

Базы данных

Тема 11

Транзакции

Управление параллельной обработкой в базах данных

- управление параллельной обработкой (управление параллелизмом, *concurrency control*) направлено на то, чтобы исключить *непредусмотренное* влияние действий одного пользователя на действия другого;
- управление параллельной обработкой, как правило, предполагает компромисс между уровнем изоляции различных операций друг от друга и производительностью системы;
- транзакция (*transaction*) является последовательностью операций, выполненных как одна логическая единица работы (*logical unit of work*), т.е. либо все действия внутри транзакции выполняются успешно, либо не выполняется ни одно из них:
 - BEGIN TRANSACTION;
 - COMMIT | ROLLBACK TRANSACTION (фиксация / откат).

Параллельная обработка транзакций

- параллельные транзакции (*parallel transactions*) – две или более транзакций, обрабатываемые одновременно;
- проблемы параллельной обработки:
 - *грязное / несогласованное чтение* (dirty / inconsistent reads);
 - *невоспроизводимое / фантомное чтение* (unrepeatable / phantom reads);
 - *потерянное обновление* (lost / concurrent update);
- необходимо упорядочить операции внутри транзакций таким образом, чтобы предотвратить *нежелательное влияние* одной транзакции на другую.

Проблема грязного чтения

- грязное чтение (dirty read)

Write-Read Conflict

T_1 : WRITE(A)

T_1 : ABORT

T_2 : READ(A)

Проблема несогласованного чтения

- несогласованное чтение (read skew)

Write-Read Conflict

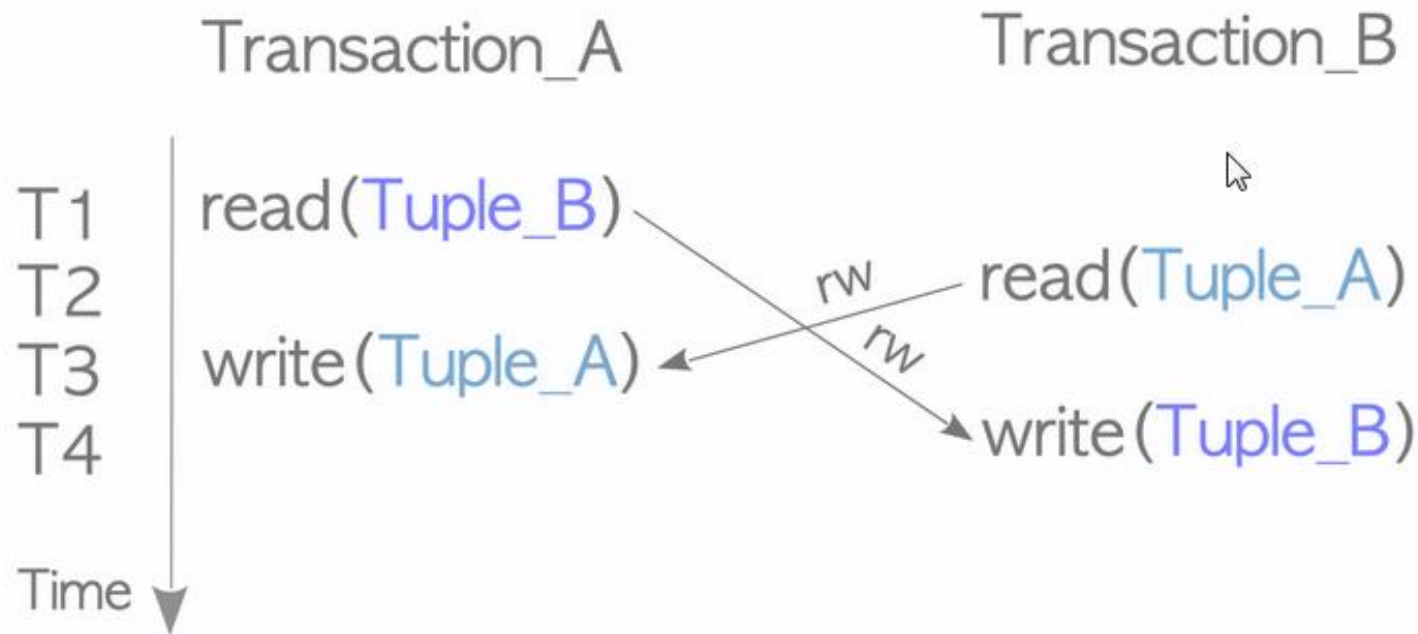
T_1 : $A := 20$; $B := 20$;
 T_1 : WRITE(A)

T_1 : WRITE(B)

T_2 : READ(A);
 T_2 : READ(B);

Проблема несогласованной записи

- несогласованная запись (write skew)



Проблема невоспроизводимого чтения

- невоспроизводимое чтение
(non-repeatable read)

Read-Write Conflict

T_1 : WRITE(A)

T_2 : READ(A);

T_2 : READ(A);

Проблема потерянного обновления

User A

1. Read item 100
(item count is 10).
2. Reduce count of items by 5.
3. Write item 100.

User B

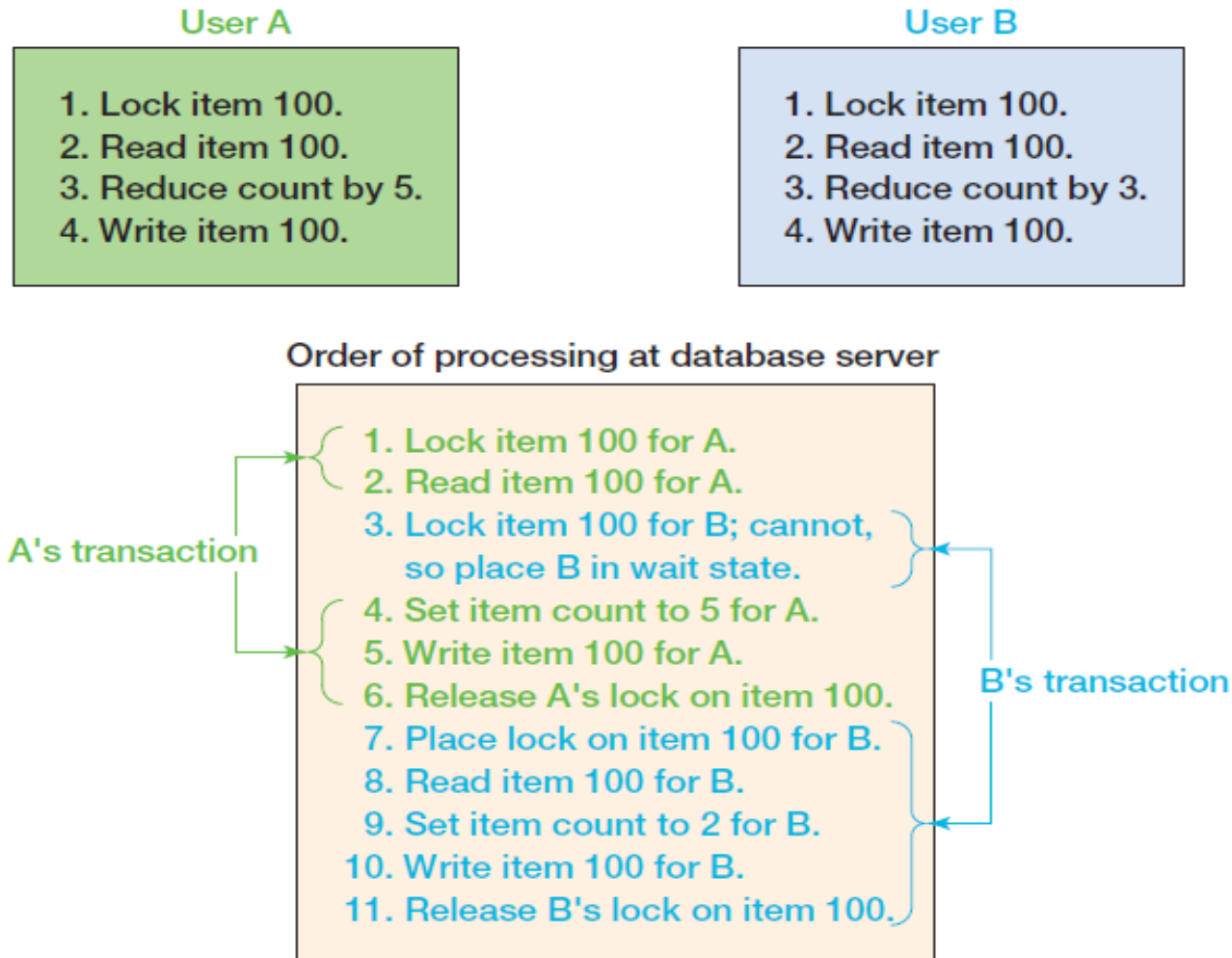
1. Read item 100
(item count is 10).
2. Reduce count of items by 3.
3. Write item 100.

Order of processing at database server

1. Read item 100 (for A).
2. Read item 100 (for B).
3. Set item count to 5 (for A).
4. Write item 100 for A.
5. Set item count to 7 (for B).
6. Write item 100 for B.

Note: The change and write in steps 3 and 4 are lost.

Решение проблемы потерянного обновления с помощью транзакций



Сериализуемость транзакций

- идеальный случай: последовательное (*serial*) выполнение, при котором исход транзакции зависит только от состояния данных перед её началом и от действий внутри самой транзакции;
- транзакция – последовательность операций чтения и записи (неделимые абстрактные операции над объектами базы данных);
- конфликт двух операций возникает, если:
 - они обращаются к одному и тому же объекту;
 - хотя бы одна из них является операцией записи;
- последовательности операций (*schedule*) являются эквивалентными относительно конфликтов (*conflict equivalence*), если конфликтующие операции выполняются в одинаковом порядке;
- сериализуемость транзакций (*serializable transactions*) – такая схема обработки транзакций, при которой результат их параллельной обработки такой же, как если бы они выполнялись последовательно (то есть, существует эквивалентная относительно конфликтов последовательность операций, соответствующая последовательному выполнению транзакций).

Свойства транзакций

- транзакция обладает четырьмя свойствами (ACID):
 - *атомарность (atomicity)*: транзакция должна быть атомарной единицей работы; должны быть выполнены либо все входящие в нее модификации данных, либо ни одно из них не должно быть выполнено;
 - *согласованность (consistency)*: по завершении транзакция должна оставить все данные в согласованном состоянии (включая все внутренние структуры данных), для этого модификации должны применяться к данным в том состоянии, в котором они были на момент начала операции:
 - на уровне оператора (*statement-level consistency*);
 - на уровне транзакции (*transaction-level consistency*);
 - *изоляция (isolation)*: выполняемые транзакцией модификации должны быть (могут быть) изолированы от любых модификаций, проводимых другими транзакциями;
 - *устойчивость (durability)*: после завершения транзакции произведенные ею действия фиксируются в системе (даже в случае системного сбоя).

Согласованность транзакций

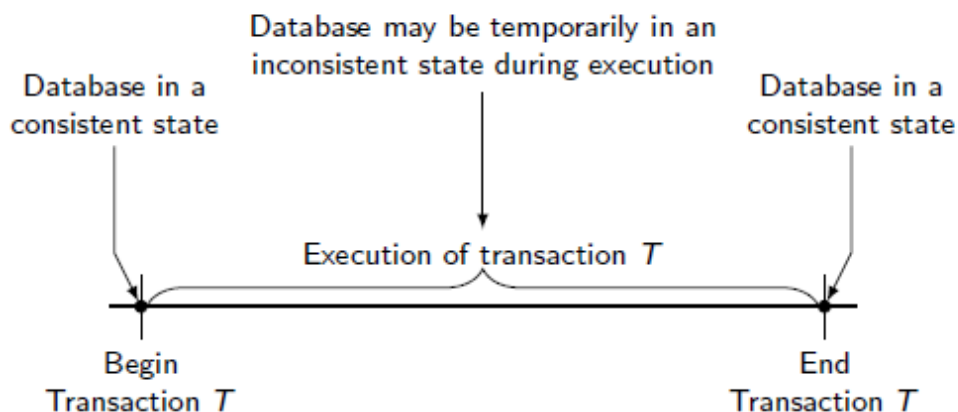
Согласованность (consistency): по завершении транзакция должна оставить все данные в согласованном состоянии (включая все внутренние структуры данных), для этого модификации должны применяться к данным в том состоянии, в котором они были на момент начала операции:

- на уровне оператора (*statement-level consistency*);
- на уровне транзакции (*transaction-level consistency*).

```
BEGIN TRANSACTION;  
UPDATE CUSTOMER  
    SET AreaCode = '425'  
    WHERE ZipCode = '98050';  
COMMIT TRANSACTION;
```

```
BEGIN TRANSACTION;  
UPDATE CUSTOMER  
    SET AreaCode = '425'  
    WHERE ZipCode = '98050';
```

```
UPDATE CUSTOMER  
    SET Discount = 0.05  
    WHERE AreaCode = '425';  
COMMIT TRANSACTION;
```



Изоляция транзакций

- цель изоляции транзакций (сериализации) – предотвращение чтения неполных результатов другими транзакциями, что предотвращает появление различных аномалий (таких как потерянные обновления и т.п.) и необходимость каскадных откатов (*cascade aborts*);
- управление параллельной обработкой, как правило, предполагает компромисс между уровнем изоляции различных операций друг от друга и производительностью системы;
- фактически изоляция транзакций означает, что клиент СУБД может устанавливать требуемый уровень изоляции, а СУБД, соответственно, обеспечивает управление параллельным выполнением для достижения нужного уровня изоляции.

Стандарт SQL-92: аномалии изоляции транзакций

- проблемы (аномалии, *phenomena*) изоляции транзакций, рассматриваемые стандартом SQL-92:
 - «грязное» чтение (*dirty read*) – чтение транзакцией записи, измененной другой транзакцией, при этом эти изменения еще не зафиксированы;
 - невозпроизводимое чтение (*non-repeatable read*) – при повторном чтении транзакция обнаруживает измененные или удаленные данные, зафиксированные другой транзакцией;
 - фантомное чтение (*phantom read*) – при повторном чтении транзакция обнаруживает новые строки, вставленные другой завершенной транзакцией;

Стандарт SQL-92: уровни изоляции транзакций

- стандарт SQL-92 определяет четыре уровня изоляции транзакций:
 - незавершенное чтение (*read uncommitted*);
 - завершенное чтение (*read committed*);
 - воспроизводимое чтение (*repeatable read*);
 - сериализуемость (*serializable [anomaly-serializable]*);

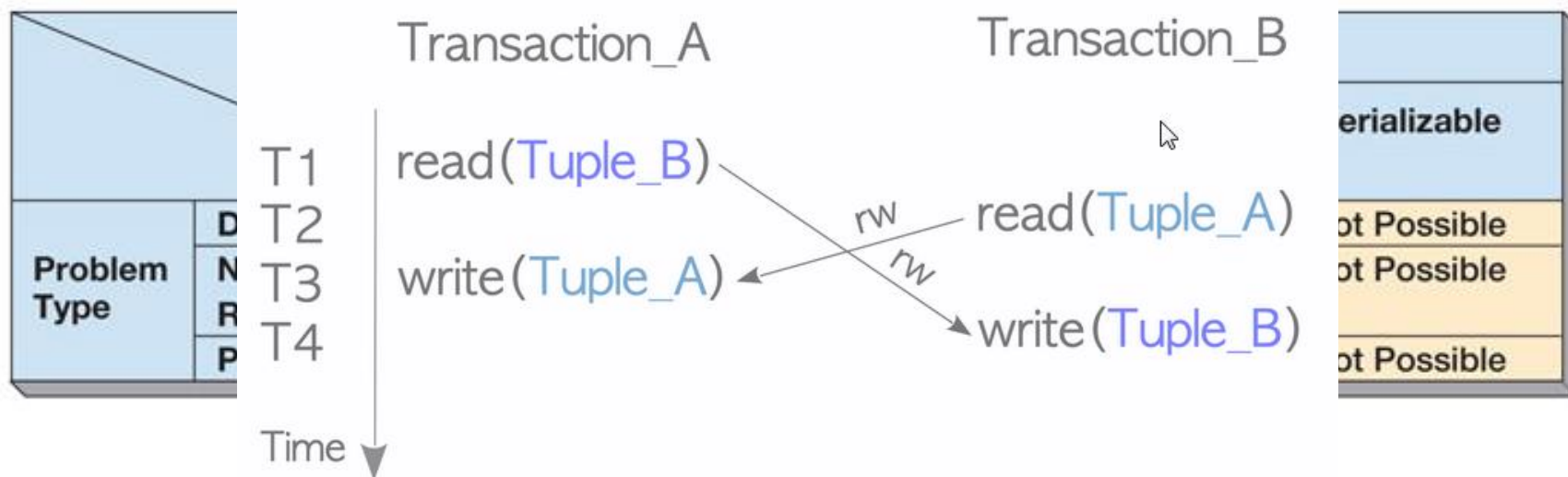
		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not Possible	Not Possible	Not Possible
	Nonrepeatable Read	Possible	Possible	Not Possible	Not Possible
	Phantom Read	Possible	Possible	Possible	Not Possible

Платформы и уровни изоляции

	Read uncommitted	Read committed	Cursor stability	Repeatable read	Snapshot isolation	Serializable
Oracle		+				+
Db2	+		+	+		+
Microsoft SQL Server	+	+		+	+	+
PostgreSQL	+	+		+		+
MySQL	+	+		+		+
MongoDB ²	+	+			+	
SQLite					+	
CockroachDB						+
YugabyteDB		+			+	+

Изоляция моментальных снимков

- изоляция моментальных снимков (snapshot isolation) позволяет транзакциям получать согласованный набор данных, существовавший на момент начала транзакции – версионирование строк (*row versioning*);
- таким образом, чтение данных никогда не блокируется записью, но данные могут быть неактуальными;
- фиксация транзакции происходит только если изменяемые данные не были изменены (зафиксированы) другой транзакцией.

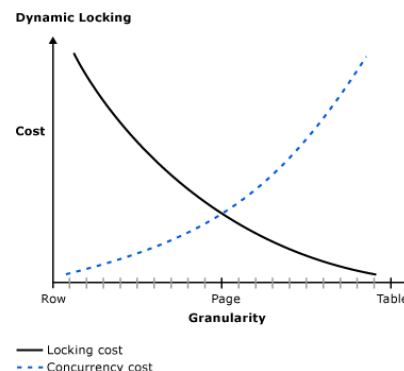


Стратегии и алгоритмы управления параллелизмом

- механизмы синхронизации:
 - блокировка (*resource-locking*): взаимоисключающий доступ к разделяемым данным;
 - упорядочивание транзакций в соответствии с набором правил (*transaction ordering*), в том числе упорядочивание по временным меткам (*TO, timestamp ordering*);
- стратегии, обеспечивающие изоляцию транзакций:
 - пессимистическая (*pessimistic concurrency control*): обнаружение и предотвращение конфликтов в процессе выполнения транзакции;
 - оптимистическая блокировка (*optimistic concurrency control*): анализ конфликтов в момент фиксации транзакции.

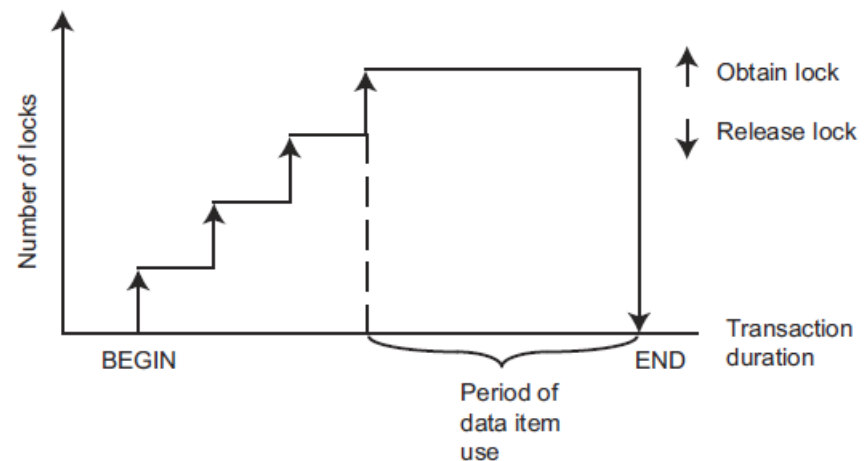
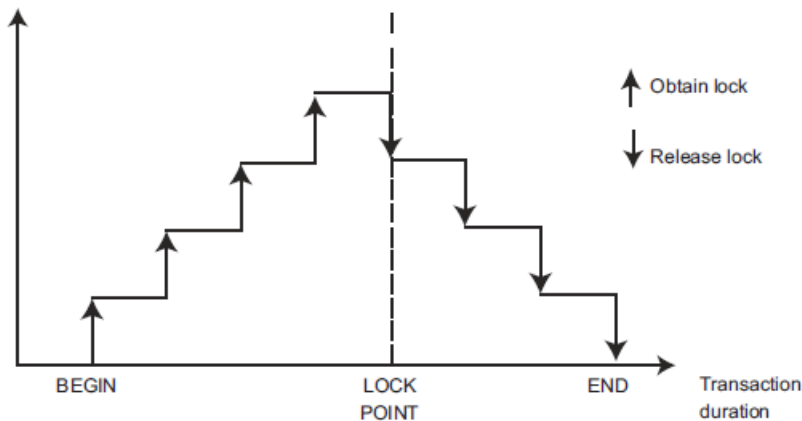
Блокировка ресурсов

- инициатор блокировки:
 - СУБД: неявная блокировка (*implicit lock*);
 - программа / запрос: явная блокировка (*explicit lock*);
- степень (глубина) детализации блокировки (*locking granularity*):
 - на уровне базы данных;
 - на уровне таблицы;
 - на уровне страницы;
 - на уровне строки;
- режим блокировки (*lock mode*):
 - *монопольный* (*exclusive lock / write lock*): блокируются все виды доступа к элементу;
 - *разделяемый* (*shared lock / read lock*): блокируется только запись.



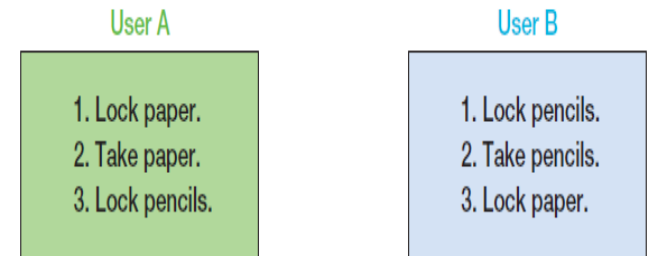
Двухфазная блокировка

- в общем случае типы и длительность удержания блокировок определяют уровень изоляции транзакций;
- один из способов достижения *сериализуемости* – использование *двухфазной блокировки* (two-phase locking):
 - *фаза нарастания* (growing phase): транзакция может налагать блокировки по мере необходимости, до тех пор, пока не снимается хотя бы одна блокировка;
 - *фаза сжатия* (shrinking phase): после снятия одной из блокировок никакие другие накладываться уже не могут;
- модификация двухфазной блокировки (S2PL – strict 2PL): снятие блокировок непосредственно в момент фиксации / отката транзакции (предотвращает каскадные откаты).

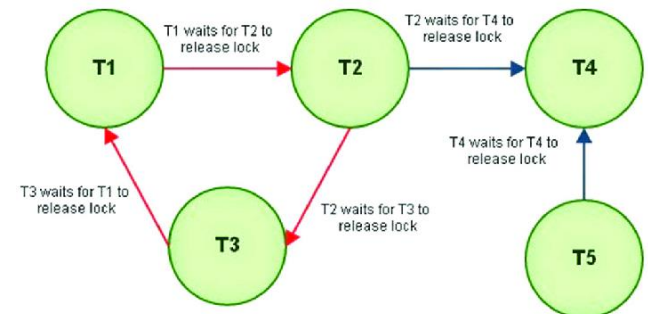
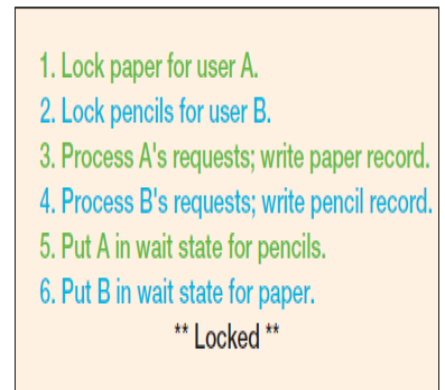


Взаимная блокировка

- взаимная блокировка (*deadlock, deadly embrace*) – состояние взаимного ожидания транзакциями освобождения ресурсов, заблокированных друг другом;
- анализ: граф ожидания (*WFG, wait-for graph*);
- обработка взаимной блокировки:
 - предотвращение (*deadlock prevention*): блокирование всех ресурсов одновременно в начале операции;
 - избежание (*deadlock avoidance*): требование блокировки в одинаковом порядке;
 - обнаружение и устранение (*deadlock detection and resolution*): откат одной или нескольких транзакций, вызвавших блокировку (*victim transaction*) – оценка стоимости отката / стоимости завершения / количества разрешаемых блокировок (циклов).



Order of processing at database server



Упорядочивание транзакций

- алгоритмы упорядочивания транзакций выбирают порядок сериализации априори – в соответствии с уникальной и монотонно возрастающей меткой транзакции (*transaction timestamp*);
- правило упорядочивания: конфликтующая операция более «старой» транзакции имеет приоритет;
- при наличии конфликтов в очередь операций может быть добавлена новая операция только той транзакции, которая является самой последней из конфликтующих; в противном случае производится откат / перезапуск транзакции;
- для реализации данного алгоритма каждый объект должен также иметь метки чтения и записи, соответствующие максимальным меткам транзакций, осуществлявшим чтение / запись данного объекта.

Многоверсионное управление параллельным доступом (MVCC, Multi-Version Concurrency Control)

- множественные версии объекта («снимки», *snapshots*);
- завершённое чтение (read committed):
 - при чтении транзакция получает самую последнюю версию объекта *на момент начала очередной инструкции*;
- изоляция моментальных снимков (SI, snapshot isolation):
 - при чтении транзакция получает самую последнюю версию объекта *на момент начала данной транзакции*;
 - фиксация транзакции происходит только если изменяемые данные не были изменены (зафиксированы) другой транзакцией (“first committer wins”);
- сериализуемая изоляция моментальных снимков (SSI, serializable snapshot isolation):
 - при чтении транзакция получает самую последнюю версию объекта *на момент начала данной транзакции*;
 - фиксация транзакции происходит только если изменяемые данные не были изменены (зафиксированы) или прочитаны другой транзакцией;
- собственные изменения видны в транзакции.

Многоверсионное управление параллельным доступом: сериализуемая изоляция моментальных снимков

- множественные версии объекта («снимки»);
- при чтении транзакция получает самую последнюю версию объекта *на момент начала данной транзакции*;
- при записи:

$$TS(T_i) < RTS(X_k) = TS(T_j) \Rightarrow cancel T_i$$

$$X_k^t \rightarrow X_k^{t+1}; RTS(X_k^{t+1}) = WTS(X_k^{t+1}) = TS(T_i)$$

- условие удаления «устаревших» версий:

$$\exists X_k^{t+1} | TS(X_k^{t+1}) > TS(X_k^t) \& \forall T_i TS(T_i) > TS(X_k^{t+1})$$



Уровни изоляции в популярных реляционных СУБД

 PostgreSQL	Dirty Read	Lost Update	Phantom	Nonrepeatable read	Skewed write
Read Uncommitted	No	Yes	Yes	Yes	Yes
Read Committed					
Repeatable Read	No	No	No	No	Yes
Serializable	No	No	No	No	No

 Microsoft SQL Server	Dirty Read	Lost Update	Phantom	Nonrepeatable read	Skewed write
Read Uncommitted	Yes	Yes	Yes	Yes	Yes
Read Committed	No	Yes	Yes	Yes	No
Repeatable Read	No	No	Yes	No	No
Snapshot isolation	No	No	No	No	Yes
Serializable	No	No	No	No	No

ORACLE	Dirty Read	Lost Update	Phantom	Nonrepeatable read	Skewed write
Read Uncommitted	-				
Read Committed	No	Yes	Yes	Yes	Yes
Repeatable Read	-				
Serializable	No	No	No	No	Yes

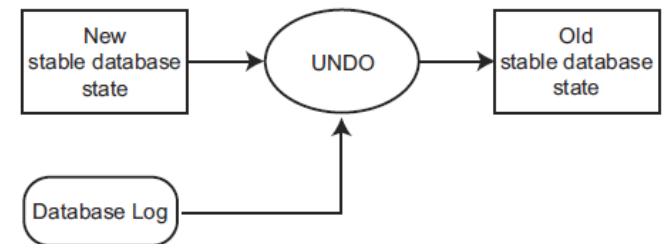
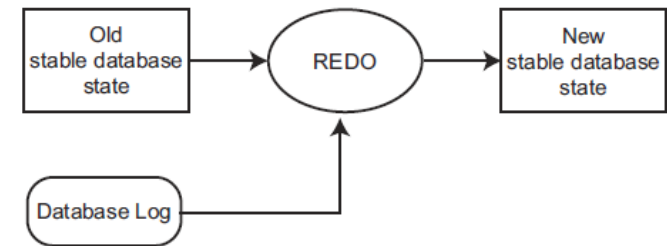
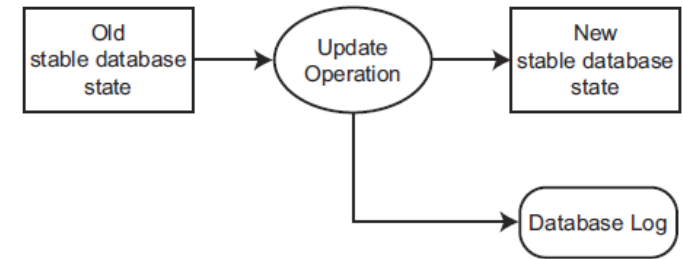
 MySQL	Dirty Read	Lost Update	Phantom	Nonrepeatable read	Skewed write
Read Uncommitted	Yes	Yes	Yes	Yes	Yes
Read Committed	No	Yes	Yes	Yes	Yes
Repeatable Read	No	Yes	No	No	Yes
Serializable	No	No	No	No	No

Восстановление баз данных

- три типа сбоев в базах данных:
 - сбои транзакций (*transaction failures*) – откаты;
 - системные сбои (*system failures*) – потеря содержимого оперативной памяти;
 - сбои носителей (*media failures*) – потеря данных на диске;
- для предотвращения катастрофических последствий при сбое носителей:
 - дублирование дисков;
 - создание резервных копий базы данных (полных или инкрементных копий);
- восстановление после сбоев транзакций и системных сбоев зависит от организации записи обновлений основных данных в СУБД:
 - обновление с перезаписью (*in-place updating*);
 - обновление с заменой (*out-of-place updating*);
- для восстановления в СУБД с перезаписью необходим журнал базы данных (*database log*):
 - синонимы: журнал транзакций (*transaction log*), журналом завершённых транзакций (*commit log*), журнал повторного выполнения (*redo log*) или журнал опережающей записи (*write-ahead log*, *WAL*);
 - содержит информацию для восстановления (*recovery information*), которая необходима для выполнения набора идемпотентных операций.

Восстановление баз данных: подходы

- повторная обработка (reprocessing):
периодические снимки базы данных и записи о всех операциях со времени последнего сохранения резервной копии – нереализуемый подход в силу отсутствия времени и асинхронного характера первоначальных изменений;
- откат-накат (rollback-rollforward):
 - периодические снимки базы данных и журнал транзакций со времени последнего сохранения;
 - rollforward: выполнение всех корректных транзакций (для повторного выполнения транзакции журнал должен содержать копию записей базы данных после их изменения – конечные образы, *after-images*);
 - rollback: отмена изменений, вызванных некорректными или незавершенными транзакциями (для отмены транзакции журнал должен содержать копию записей базы данных до их изменения – исходные образы, *before-images*);



СУБД SQL Server 2025 – Транзакции и восстановление.

Управление параллельной обработкой

- определение уровня изоляции транзакции:

```
SET TRANSACTION ISOLATION LEVEL
```

```
{ READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ |  
  SNAPSHOT | SERIALIZABLE }
```

- определение характеристик курсора:

```
DECLARE cursor_name CURSOR
```

```
[ LOCAL | GLOBAL ]
```

```
[ FORWARD_ONLY | SCROLL ]
```

```
[ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
```

```
[ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
```

```
[ TYPE_WARNING ]
```

```
FOR select_statement
```

```
[ FOR UPDATE [ OF column_name [ ,...n ] ] ]
```

- блокировочные подсказки инструкций (SELECT, INSERT, UPDATE, DELETE):

```
SELECT ... FROM ... WITH
```

Определение уровня изоляции транзакций

- возможные уровни изоляции (блокировка / версионирование):
 - READ UNCOMMITTED;
 - **READ COMMITTED;**
 - READ_COMMITTED_SNAPSHOT (ON | OFF);
 - REPEATABLE READ;
 - SNAPSHOT – согласованность на уровне транзакции;
 - SERIALIZABLE;
- возможно переключение уровня изоляции внутри транзакции (кроме → SNAPSHOT, правила блокировок применяются на основании текущего уровня изоляции);
- при выходе из хранимой процедуры / триггера уровень изоляции восстанавливается до уровня, актуального на момент вызова.

		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not Possible	Not Possible	Not Possible
	Nonrepeatable Read	Possible	Possible	Not Possible	Not Possible
	Phantom Read	Possible	Possible	Possible	Not Possible

Уровни изоляции транзакций (пример)

- разделяемые блокировки удерживаются все время до завершения транзакции.

```
USE AdventureWorks;  
GO
```

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
BEGIN TRANSACTION;
```

```
SELECT * FROM HumanResources.EmployeePayHistory;  
SELECT * FROM HumanResources.Department;
```

```
COMMIT TRANSACTION;
```

Блокировочные подсказки инструкций

SELECT/UPDATE/INSERT/DELETE ... WITH (...)

- HOLDLOCK (= SERIALIZABLE)
- NOLOCK (= READUNCOMMITTED)
- READCOMMITTED
- READCOMMITTEDLOCK
- READPAST (пропуск заблокированных другими транзакциями записей)
- READUNCOMMITTED
- REPEATABLEREAD
- SERIALIZABLE
- PAGLOCK | ROWLOCK | TABLOCK
- TABLOCKX
- XLOCK

```
UPDATE Production.Product
```

```
    WITH (TABLOCK)
```

```
    SET ListPrice = ListPrice * 1.10
```

```
    WHERE ProductNumber LIKE 'BK-%'
```


Блокировочные подсказки инструкций (пример)

```
USE AdventureWorks;  
GO
```

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
BEGIN TRANSACTION;
```

```
SELECT Title FROM HumanResources.Employee WITH (NOLOCK);
```

```
-- Get information about the locks held by the transaction.  
SELECT resource_type, resource_subtype, request_mode FROM  
    sys.dm_tran_locks WHERE request_session_id = @@spid;
```

```
ROLLBACK;  
GO
```

Определение характеристик курсора

```
DECLARE cursor_name CURSOR
  [ LOCAL | GLOBAL ]
  [ FORWARD_ONLY | SCROLL ]
  [ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
  [ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
  [ TYPE_WARNING ]
  FOR select_statement
  [ FOR UPDATE [ OF column_name [ ,...n ] ] ]
```

- FORWARD_ONLY | SCROLL – возможность прокрутки курсора;
- STATIC | KEYSET | DYNAMIC | FAST_FORWARD – тип курсора;
- READ_ONLY | SCROLL_LOCKS | OPTIMISTIC (WITH VALUES / WITH ROW VERSIONING) – управление изменениями и блокировками записей;
- возможно неявное изменение типа курсора, если формирующая его инструкция выборки содержит элементы, не соответствующие требованиям курсора.

Режимы транзакций

- явные транзакции:
 - запускаются посредством инструкции `BEGIN TRANSACTION`;
 - завершение транзакции осуществляется с использованием инструкций `COMMIT TRANSACTION / WORK` (фиксация) или `ROLLBACK TRANSACTION / WORK` (откат);
 - управление транзакциями также может осуществляться через функции API – ADO, ODBC, OLEDB (`ITransactionLocal::StartTransaction()`, `ITransaction::Commit()` и `ITransaction::Abort()`);
- автоматически фиксируемые транзакции (режим по умолчанию):
 - каждая отдельная инструкция фиксируется после завершения;
- неявные транзакции:
 - режим запускается через инструкцию `SET IMPLICIT_TRANSACTIONS ON` или через соответствующие функции API (ODBC, OLEDB);
 - очередная инструкция `ALTER TABLE`, `INSERT`, `CREATE`, `OPEN`, `DELETE`, `REVOKE`, `DROP`, `SELECT`, `FETCH`, `TRUNCATE TABLE`, `GRANT`, `UPDATE` автоматически запускает новую транзакцию;
 - необходимо только фиксировать или выполнять откат каждой транзакции (`COMMIT / ROLLBACK`), после завершения транзакции следующая инструкция (из перечисленного списка) запускает новую транзакцию.

Неявные транзакции: пример (T-SQL)

```
CREATE TABLE ImplicitTran
    (Cola int PRIMARY KEY,
     Colb char(3) NOT NULL);
GO

SET IMPLICIT_TRANSACTIONS ON;

-- First implicit transaction started by an INSERT statement.
INSERT INTO ImplicitTran VALUES (1, 'aaa');
INSERT INTO ImplicitTran VALUES (2, 'bbb');

-- Commit first transaction.
COMMIT TRANSACTION;

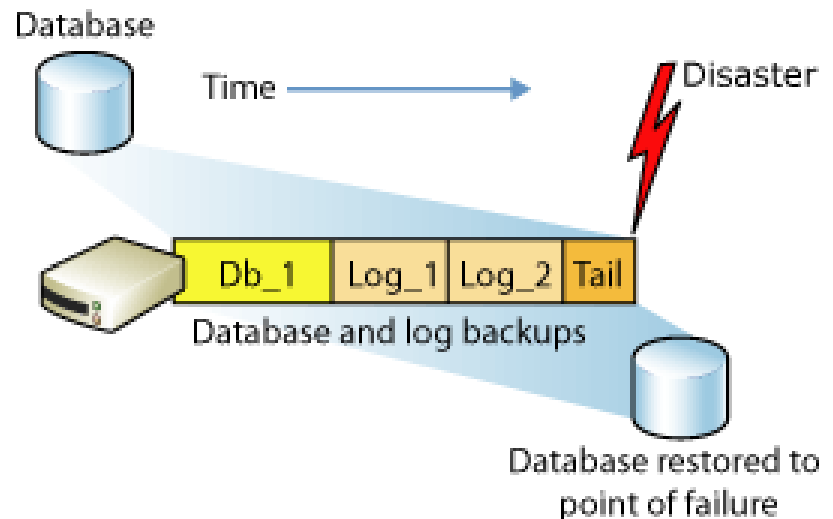
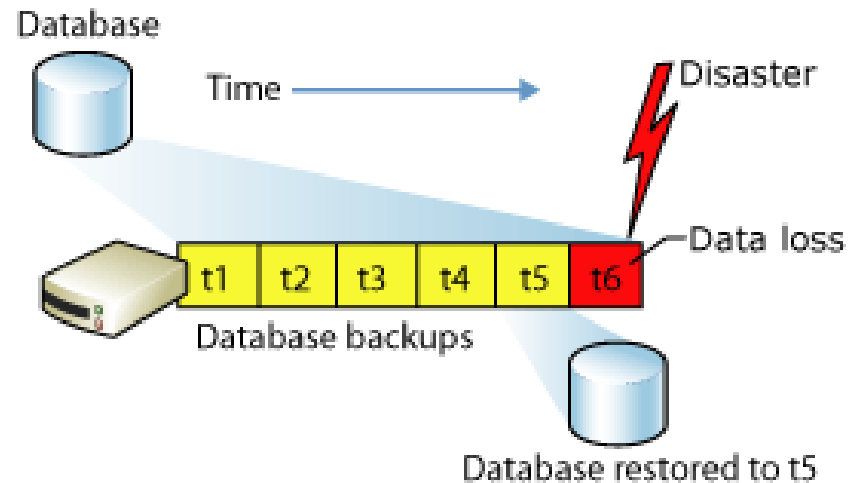
-- Second implicit transaction started by a SELECT statement.
SELECT COUNT(*) FROM ImplicitTran;
INSERT INTO ImplicitTran VALUES (3, 'ccc');
SELECT * FROM ImplicitTran;

-- Commit second transaction.
COMMIT TRANSACTION;

SET IMPLICIT_TRANSACTIONS OFF;
```

Восстановление

- создание резервной копии (данных и журнала транзакций):
 - полные (full backup);
 - частичные (differential backup);
- модели ВОССТАНОВЛЕНИЯ:
 - простая (simple recovery model);
 - полная (full recovery model);
 - частичная (bulk-logged recovery model).



Вопросы к экзамену

- Управление параллельной обработкой в БД. Проблемы (аномалии) параллельной обработки транзакций.
- Свойства транзакций. Сериализуемость транзакций.
- Стандарт SQL-92: аномалии изоляции транзакций и уровни изоляции.
- Стратегии и механизмы управления параллелизмом. Блокировка ресурсов. Алгоритм двухфазной блокировки. Взаимная блокировка.
- Стратегии и механизмы управления параллелизмом. Многоверсионное управление параллельным доступом.
- Восстановление баз данных.
- Управление параллельной обработкой в СУБД MS SQL Server.
- Транзакции в СУБД MS SQL Server. Режимы транзакций.
- Восстановление баз данных. Восстановление в СУБД MS SQL Server.

Лабораторная работа №10.

Режимы выполнения транзакций

1. Исследовать и проиллюстрировать на примерах различные уровни изоляции транзакций MS SQL Server, устанавливаемые с использованием инструкции **SET TRANSACTION ISOLATION LEVEL**
2. Накладываемые блокировки исследовать с использованием **sys.dm_tran_locks**

Лабораторная работа №11.

Создание базы данных и запросов к ней в СУБД SQL Server

1. создать базу данных, спроектированную в рамках лабораторной работы №4, используя изученные в лабораторных работах 5-10 средства SQL Server 2012:
 - поддержания создания и физической организации базы данных;
 - различных категорий целостности;
 - представления и индексы;
 - хранимые процедуры, функции и триггеры;
2. создание объектов базы данных должно осуществляться средствами DDL (CREATE/ALTER/DROP), в обязательном порядке иллюстрирующих следующие аспекты:
 - добавление и изменение полей;
 - назначение типов данных;
 - назначение ограничений целостности (PRIMARY KEY, NULL/NOT NULL/UNIQUE, CHECK и т.п.);
 - определение значений по умолчанию;

Лабораторная работа №11

(продолжение)

3. в рассматриваемой базе данных должны быть тем или иным образом (в рамках объектов базы данных или дополнительно) созданы запросы DML для:
 - выборки записей (команда SELECT);
 - добавления новых записей (команда INSERT), как с помощью непосредственного указания значений, так и с помощью команды SELECT;
 - модификации записей (команда UPDATE);
 - удаления записей (команда DELETE);
4. запросы, созданные в рамках пп.2,3 должны иллюстрировать следующие возможности языка:
 - удаление повторяющихся записей (DISTINCT);
 - выбор, упорядочивание и именование полей (создание псевдонимов для полей и таблиц / представлений);
 - соединение таблиц (INNER JOIN / LEFT JOIN / RIGHT JOIN / FULL OUTER JOIN);
 - условия выбора записей (в том числе, условия NULL / LIKE / BETWEEN / IN / EXISTS);
 - сортировка записей (ORDER BY - ASC, DESC);
 - группировка записей (GROUP BY + HAVING, использование функций агрегирования – COUNT / AVG / SUM / MIN / MAX);
 - объединение результатов нескольких запросов (UNION / UNION ALL / EXCEPT / INTERSECT);
 - вложенные запросы.